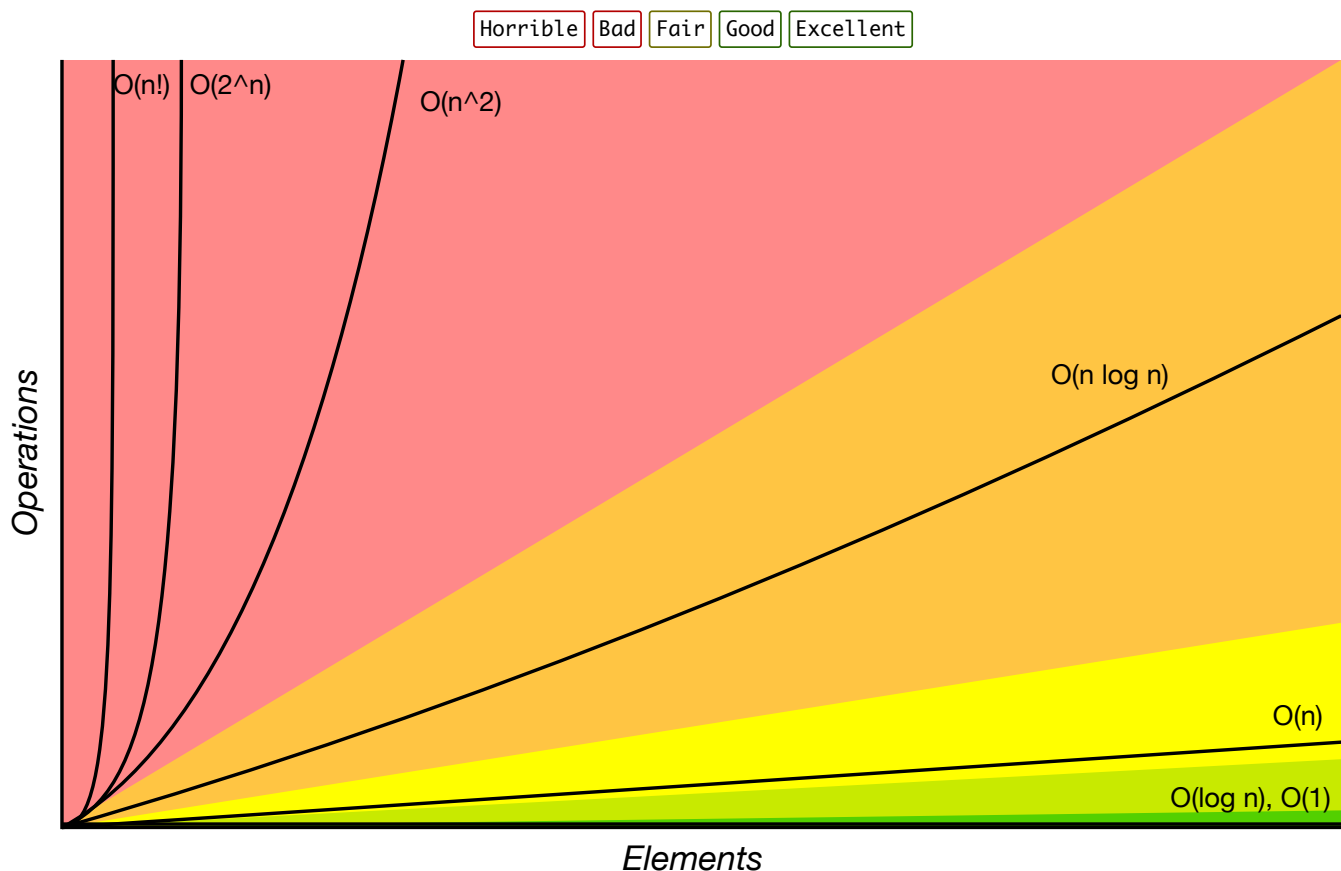


Know Thy Complexities!

Hi there! This webpage covers the space and time Big-O complexities of common algorithms used in Computer Science. When preparing for technical interviews in the past, I found myself spending hours crawling the internet putting together the best, average, and worst case complexities for search and sorting algorithms so that I wouldn't be stumped when asked about them. Over the last few years, I've interviewed at several Silicon Valley startups, and also some bigger companies, like Google, Facebook, Yahoo, LinkedIn, and Uber, and each time that I prepared for an interview, I thought to myself "Why hasn't someone created a nice Big-O cheat sheet?". So, to save all of you fine folks a ton of time, I went ahead and created one. Enjoy! - [Eric](#)

[AngularJS to React Automated Migration](#)

Big-O Complexity Chart



Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$\theta(n^2)$	$\theta(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$\theta(n \log(n))$	$\theta(n)$
<u>Timsort</u>	$\Omega(n)$	$\theta(n \log(n))$	$\theta(n \log(n))$	$\theta(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$\theta(n \log(n))$	$\theta(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\theta(n^2)$	$\theta(n^2)$	$\theta(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\theta(n^2)$	$\theta(n^2)$	$\theta(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\theta(n^2)$	$\theta(n^2)$	$\theta(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$\theta(n^2)$	$\theta(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\theta(n(\log(n))^2)$	$\theta(n(\log(n))^2)$	$\theta(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$\theta(n^2)$	$\theta(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\theta(nk)$	$\theta(nk)$	$\theta(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$\theta(n+k)$	$\theta(k)$
<u>Cubesort</u>	$\Omega(n)$	$\theta(n \log(n))$	$\theta(n \log(n))$	$\theta(n)$

Learn More

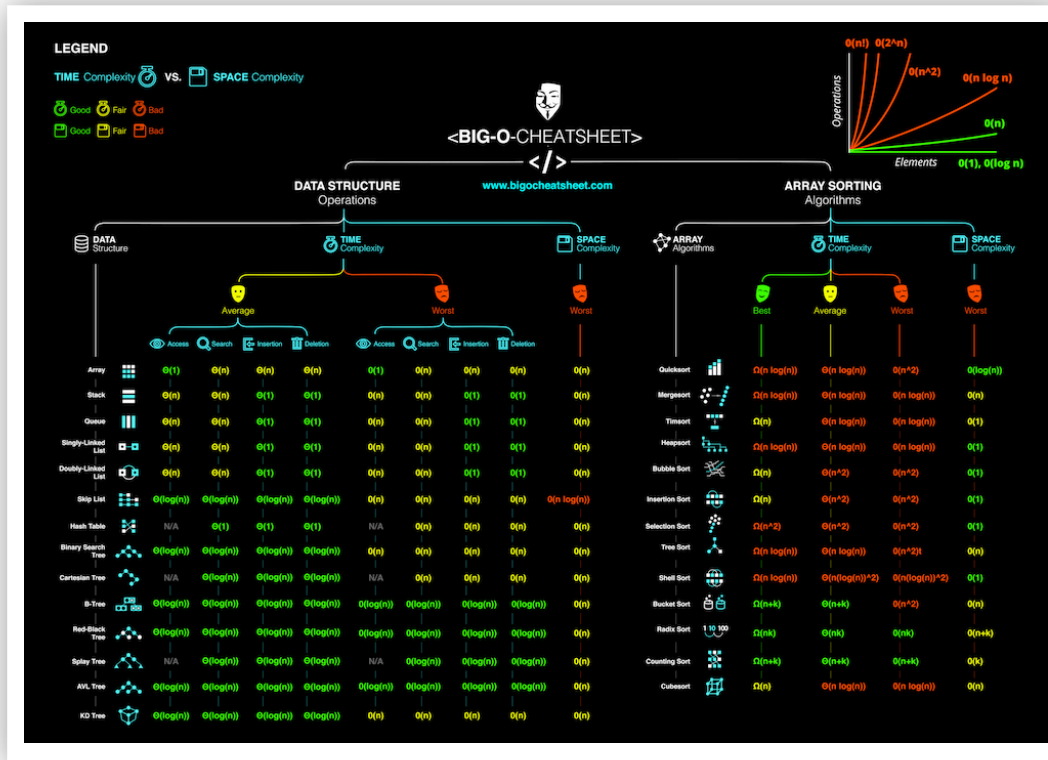
[Cracking the Coding Interview: 150 Programming Questions and Solutions](#)

[Introduction to Algorithms, 3rd Edition](#)

[Data Structures and Algorithms in Java \(2nd Edition\)](#)

[High Performance JavaScript \(Build Faster Web Application Interfaces\)](#)

Get the Official Big-O Cheat Sheet Poster



Contributors

Eric Rowell Quentin Pleple Michael Abed Nick Dizazzo Adam Forsyth Felix Zhu Jay Engineer

Josh Davis Nodir Turakulov Jennifer Hamon David Dorfman Bart Massey Ray Pereda Si Pham

Mike Davis mcverry Max Hoffmann Bahador Saket Damon Davison Alvin Wan Alan Briolat

Drew Hannay Andrew Rasmussen Dennis Tsang Vinnie Magro Adam Arold Alejandro Ramirez

Aneel Nazareth Rahul Chowdhury Jonathan McElroy steven41292 Brandon Amos Joel Friedly

Casper Van Gheluwe Eric Lefevre-Ardant Oleg Renfred Harper Piper Chester Miguel Amigot Apurva K

Matthew Daronco Yun-Cheng Lin Clay Tyler Orhan Can Ozalp Ayman Singh David Morton

Aurelien Ooms Sebastian Paaske Torholm Koushik Krishnan Drew Bailey Robert Burke

Make this Page Better

[Edit these tables!](#)

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

♡ 384

Share

Best Newest Oldest**Michael Mitchell**

13 years ago

This is great. Maybe you could include some resources (links to khan academy, mooc etc) that would explain each of these concepts for people trying to learn them.

418 1 Reply Share ›

**Amanda Harlin**

➔ Michael Mitchell

13 years ago

Yes! Please & thank you

91 1 Reply Share ›

A

Asim Ahmad

➔ Amanda Harlin

8 years ago

Can you Explain the Above Algorithm.??

0 13 Reply Share ›

A

Anonymous

➔ Asim Ahmad

8 years ago

Mr.

you can learn these algorithms easily in google by searching

Don't always ask or wait for someone to post things for you go out and search on internet

You will find everything you want to learn

If you are a beginner in Data structures and algorithms then visit mycodeschool youtube channel and learn there

if you want more then email me at rise.d1105@gmail.com I will help you as much as I can

55 17 Reply Share ›

**ichiraku demigod**

➔ Anonymous

2 years ago

this offer still goin? i got finals soon

2 0 Reply Share ›

**Biribiri**

➔ ichiraku demigod



2 years ago

Lmao

1 1 Reply Share ›

S

Safin → Anonymous

4 years ago edited

@Anonymous Your email isnt working.

0 0 Reply Share ›



Careerdrill → Anonymous

6 years ago

<https://www.careerdrill.com/>

0 3 Reply Share ›



Trey Huffine → Asim Ahmad

6 years ago

<https://skilled.dev> provides a detailed explanation

2 1 Reply Share ›

J

JSmith → Asim Ahmad

10 months ago

You're mistaken us for ChatGPT.

0 0 Reply Share ›



Cam Cecil → Michael Mitchell

13 years ago

This explanation in 'plain English' helps: <http://stackoverflow.com/qu...>

39 1 Reply Share ›



Richard Wheatley → Cam Cecil

11 years ago

this is plain english.

15 3 Reply Share ›

T

Tomás Gemes | xygrr → Richard Wheatley

5 years ago

No, this is spooky english to some you know

2 0 Reply Share ›



Arjan Nieuwenhuizen → Michael Mitchell

13 years ago edited

Here are the links that I know of.

#1) <http://aduni.org/courses/al...>

#2) <http://ocw.mit.edu/courses/...>

#3) <https://www.udacity.com/cou...>

probably as good or maybe better # 2, but I have not had a chance to look at it.

<http://ocw.mit.edu/courses/...>

Sincerely,
Arjan

p.s.

<https://www.coursera.org/co...>

This course has just begun on coursera (dated 1 July 2013), and looks very good.

23 0 Reply Share ›



fireheron

➔ Arjan Nieuwenhuizen



13 years ago

Thank you Arjan. Espaecially the coursera.org one ;-)

5 0 Reply Share ›



@hangtwentyy

➔ fireheron



11 years ago

also this! <http://opendatastructures.org>

10 0 Reply Share ›



yth

➔ @hangtwentyy



11 years ago

thank you for sharing this.

2 0 Reply Share ›



Eduardo Sánchez

➔ Michael Mitchell



10 years ago

There is an amazing tutorial for Big O form Derek Banas in Youtube, that guy is amazing explaining!!!

see more

13 1 Reply Share ›



Sudhanshu Mishra

➔ Eduardo Sánchez



10 years ago

Cool! This is a more than adequate introduction! Thanks a ton for sharing!

3 0 Reply Share ›

M

Mohammed Hameed

→ Eduardo Sánchez

7 years ago

Thanks...

0 0 Reply Share ›

**CodeMunkey**

→ Michael Mitchell

10 years ago

Not sure if this helps, but here's a more visual learner for some of these algorithms - if you're interested.
<http://visualgo.net>

6 0 Reply Share ›

**Trey Huffine**

→ Michael Mitchell

6 years ago

<https://skilled.dev/> for a detailed explanation on Big O

2 0 Reply Share ›

**Divyendra Patil**

→ Michael Mitchell

8 years ago

www.codenza.us

2 0 Reply Share ›

**Ahmed KHABER**

→ Michael Mitchell

5 years ago

<https://classroom.udacity.c...>

1 0 Reply Share ›

**nate lipp**

→ Michael Mitchell

9 years ago

This is a well put together introduction
<https://www.interviewcake.c...>

1 0 Reply Share ›

Q

Qudratjon Komilov

→ Michael Mitchell

a year ago

bro you really wrote this message 12 years ago, reply to it if you are still alive

0 0 Reply Share ›

S

Sam

→ Michael Mitchell

3 years ago

Here's another great article on the most common sorting algorithms in python:

<https://educashare.com/most...>

0 0 Reply Share ›

P

Pranati B

→ Michael Mitchell

3 years ago

This post is really great. If you are looking for a more detailed explanation with code, you can check out this link.

<https://botbark.com/2023/01...>

0 0 Reply Share ›



Abby Jones

➔ Michael Mitchell

7 years ago

Fabulous idea!

0 0 Reply Share ›



Jessica Morneau

➔ Michael Mitchell

7 years ago

Or you could have supplied them in your comment instead.

0 0 Reply Share ›

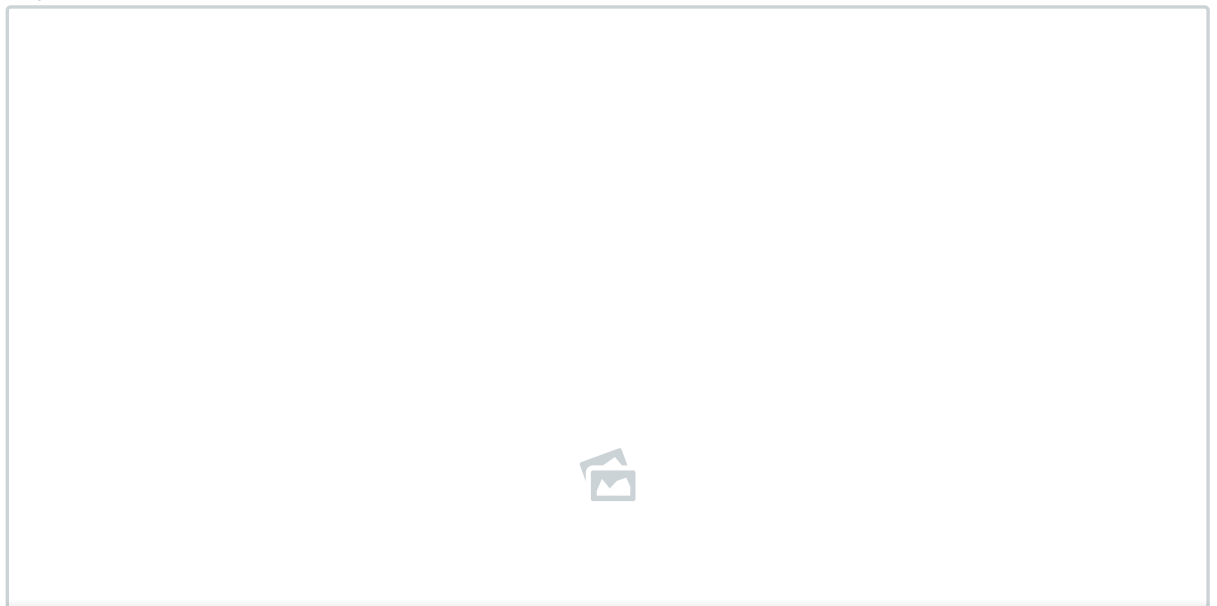
N

Nhập Hàng Ngoại

➔ Michael Mitchell

9 years ago

<http://fashionfor.life/t-sh...>



see more

0 16 Reply Share ›



Blake Jennings

13 years ago

i'm literally crying

102 0 Reply Share ›

F

friend

➔ Blake Jennings

9 years ago

you give me a big o

6 0 Reply Share ›



Gokce Toykuyu

14 years ago

Could we add some tree algorithms and complexities? Thanks. I really like the Red-Black trees ;)

91 0 Reply Share ›



ericdrowell Mod

→ Gokce Toykuyu

14 years ago

Excellent idea. I'll add a section that compares insertion, deletion, and search complexities for specific data structures

31

0

Reply Share ›

Y

yash bedi

→ ericdrowell

9 years ago

its been 4 years you haven't added that section :)

0

1

Reply Share ›



Elliot Géhin

→ yash bedi

9 years ago

It's up there Yash, bottom of the first table

1

0

Reply Share ›

A

Ahsan Sukamuljo

→ yash bedi

6 years ago

fuck off asshole

0

7

Reply Share ›



Valentin Stanciu

13 years ago

1. Deletion/insertion in a single linked list is implementation dependent. For the question of "Here's a pointer to an element, how much does it take to delete it?", single-linked lists take $O(N)$ since you have to search for the element that points to the element being deleted. Double-linked lists solve this problem.
2. Hashes come in a million varieties. However with a good distribution function they are $O(\log N)$ worst case. Using a double hashing algorithm, you end up with a worst case of $O(\log \log N)$.
3. For trees, the table should probably also contain heaps and the complexities for the operation "Get Minimum".

63

0

Reply Share ›



This comment was deleted.



Miguel

→ Guest

11 years ago

You still have to find the position in the list, which can only be done linearly.

8

0

Reply Share ›

G

Guest → Miguel

11 years ago edited

You still have to find the position in the list, which can only be done linearly.

3

0

Reply Share ›



This comment was deleted.

G

Guest → Guest

11 years ago

No need to find the position if you can delete it as Alexis mentioned

2 1 Reply Share ›

P

Pingu App

→ Guest

11 years ago

What if B is the last element in the list?

How would B's predecessor know that its next field should point to NULL and not to a futurely invalid memory address?

3 0 Reply Share ›

D

Da Zeagra Oil

→ Pingu App

4 months ago

that's confusing, so if u want to delete the last element of the list, to maintain the delete method working u should receive two params, node to delete and previous node?

<https://dazeagraoil.com/>

0 0 Reply Share ›



This comment was deleted.

O

Oscar Martinez Sanchez

→ Guest

6 years ago

that's confusing, so if u want to delete the last element of the list, to maintain the delete method working u should receive two params, node to delete and previous node?

1 0 Reply Share ›



pvlbzn → Guest

9 years ago edited

And you will introduce the side effect which will be hell to debug. Consider:

Singly linked list { A:1, B:2, C:3, D:4 } where is X:Y, y is a value, function `delete` which works as you described, function `get` which returns pointer to the node by index.

...

```
// Take needed node C
node_t* node = get(list, 2)
print(node->value) // prints 3
```

```
// Delete B
delete(list, 1)
```

```
// Trv to access C again
```